UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE
DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN

INFORMATICA

PROGETTO D'ESAME DI

OBJECT ORIENTATION

PROGETTAZIONE E SVILUPPO DI UN APPLICATIVO IN JAVA PER LA GESTIONE DI TO-DO PERSONALI E COLLABORATIVI

Docente: Prof. Porfirio Tramontana

Studenti: Benedetta Nalbone (N86005178),
Gabriella Scaraglia (N86005338)

Anno Accademico 2024-2025

Indice

1 Introduzione al Progetto **3**

1.1 Contesto e Scopo dell'Applicazione 3

1.2 Requisiti e Funzionalità Core 3

1.3 Repository del Codice Sorgente 4

2 Analisi dei Requisiti e Casi d'Uso **5**

2.1 Specifica dei Casi d'Uso Principali 5

2.1.1 UC1: Autenticazione e Inizializzazione 5

2.1.2 UC2: Gestione del Ciclo di Vita di un ToDo 5

2.1.3 UC3: Condivisione multi-utente 6

3 Modello del Dominio **7**

3.1 Diagramma Concettuale 7

3.2 Analisi delle Entità e delle Enumerazioni 7

4 Architettura della Soluzione **9**

4.1 Diagramma UML dell'Applicativo 9

4.2 Applicazione del Pattern MVC 9

4.3 Pattern DAO e Singleton 10

5 Progettazione della Base di Dati **11**

5.1 Schema Logico Relazionale 11

5.2 Analisi delle Scelte Architetturali lato DB 11

6 CRC Cards e Relazioni **13**

6.1 CRC Cards del Dominio 13

6.2 CRC Cards della Soluzione Architetturale (MVC/DAO) 14

6.3 Analisi Dettagliata delle Relazioni UML 14

7 Dettagli Implementativi (Listings) **16**

7.1 Regole di Business Temporali (24h Rule) 16

7.2 Calcolo Dinamico del Contrasto Cromatico 16

7.3 Interattività Drag & Drop con TransferHandler 17

7.4 Gestione dei Permessi e Sicurezza lato View 18

8 Conclusioni **19**
8.1 Analisi dei Risultati 19
8.2 Qualità del Codice e Manutenibilità 19
8.3 Prospettive di Sviluppo Futuro 19

1. Introduzione al Progetto

1.1 Contesto e Scopo dell'Applicazione

L'obiettivo del presente progetto è la progettazione e l'implementazione di un'applicazione desktop in Java dedicata alla gestione avanzata di attività (ToDo List). Il sistema è stato concepito per superare i limiti di un semplice task manager individuale, integrando logiche di organizzazione per bacheche e, soprattutto, meccanismi di collaborazione multi-utente sicuri e controllati. L'applicativo si appoggia su un'interfaccia grafica intuitiva realizzata nativamente tramite il framework Java Swing e garantisce la persistenza dei dati attraverso l'integrazione con un database relazionale PostgreSQL.

1.2 Requisiti e Funzionalità Core

Il software sviluppato copre un ampio spettro di funzionalità, mappate sulle esigenze di un utente che necessita di gestire task accademici, lavorativi e personali:

- **Autenticazione Sicura:** Sistema di registrazione e login con meccanismi di hashing per la protezione delle credenziali nel database.
- **Gestione Strutturata (Bacheche):** Assegnazione immediata di tre bacheche predefinite alla registrazione, utilizzabili come contenitori logici per ordinare i ToDo.
- **Ciclo di Vita del ToDo:** Possibilità di creare, modificare e marcare come completate le attività. Ogni ToDo è arricchito da scadenze temporali, descrizioni, link URL multipli, immagini allegate e un colore identificativo personalizzato.
- **Condivisione e Permessi:** Un utente può invitare altri account a collaborare su specifici ToDo, decidendo se conferire un accesso in sola lettura o diritti di modifica, gestito tutto in tempo reale.
- **Interattività Avanzata (Drag & Drop):** L'interfaccia supporta il riposizionamento dinamico delle schede (ToDoCard) all'interno delle bacheche tramite trascinamento (drag and drop), aggiornando l'indice di posizione sul database in background.

1.3 Repository del Codice Sorgente

L'intero ciclo di sviluppo, il tracciamento delle modifiche, il refactoring e la gestione delle versioni del codice sono stati curati tramite il sistema di versionamento distribuito Git. Il repository contenente il codice sorgente completo, la documentazione Javadoc generata, le risorse grafiche e gli script SQL di configurazione del database è accessibile pubblicamente al seguente indirizzo:
`https://github.com/BenniNlb/Gestore-ToDo.git`

2. Analisi dei Requisiti e Casi d'Uso

Al fine di tracciare correttamente i requisiti funzionali del sistema, sono stati definiti i principali Casi d'Uso (Use Cases) che descrivono le interazioni tra l'unico attore del sistema (l'Utente Autenticato) e l'applicativo.

2.1 Specifica dei Casi d'Uso Principali

2.1.1 UC1: Autenticazione e Inizializzazione

Attore Primario	Utente non autenticato
Precondizione	L'applicazione è avviata e connessa al database.
Flusso Principale	1. L'utente accede alla `RegisterView`. 2. Inserisce username e password. 3. Il sistema delega al `RegisterController` la cifratura della password. 4. Il sistema genera automaticamente tre bacheche di default (UNIVERSITÀ, LAVORO, TEMPO_LIBERO) per il nuovo utente.
Postcondizione	L'utente è registrato, autenticato e reindirizzato alla `BoardView` popolata con le bacheche iniziali.

2.1.2 UC2: Gestione del Ciclo di Vita di un ToDo

Attore Primario	Utente Autenticato
Precondizione	L'utente visualizza almeno una Bacheca.
Flusso Principale	1. L'utente clicca su "Aggiungi ToDo" in una bacheca. 2. Compila il form inserendo titolo (obbligatorio), data scadenza, colore, descrizione, link e immagine. 3. Il `ToDoController` salva l'oggetto e aggiorna la UI. 4. L'utente può contrassegnare il ToDo come completato o trascinarlo per riordinarlo.
Eccezioni / Vincoli	Se la data di scadenza del ToDo è superata da oltre 24 ore, l'interfaccia ne inibisce la modifica e il riposizionamento (ReadOnly).

2.1.3 UC3: Condivisione multi-utente

Attore Primario	Utente Autenticato (Autore del ToDo)
Precondizione	L'utente ha selezionato un ToDo di sua creazione.
Flusso Principale	1. L'autore apre il pannello "Condivisioni". 2. Cerca un altro utente tramite username. 3. Assegna un livello di permesso (`SOLO_LETTURA` o `MODIFICA`). 4. Il sistema registra l'associazione.
Postcondizione	L'utente invitato, al successivo login, visualizzerà il ToDo condiviso sulla propria bacheca, con le interazioni grafiche limitate dal permesso ricevuto.

3. Modello del Dominio

3.1 Diagramma Concettuale

Il Domain Model astrae completamente i dettagli tecnologici (linguaggio di programmazione, librerie grafiche o tabelle SQL) per concentrarsi esclusivamente sulle entità logiche che compongono il problema e sulle loro interazioni.

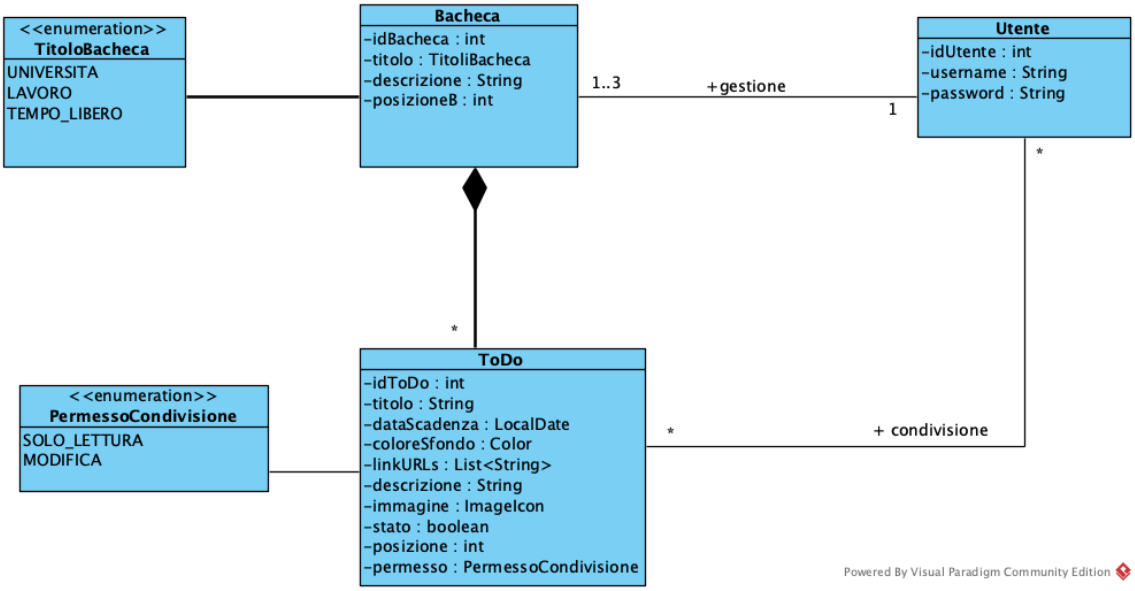

Figura 3.1: UML - Diagramma delle Classi del Dominio

3.2 Analisi delle Entità e delle Enumerazioni

Il dominio poggia su tre entità fondamentali e due enumerazioni di supporto:

- **Utente:** L'attore principale del sistema, identificato univocamente da un `idUtente`, uno `username` e una `password`.
- **Bacheca:** Il raggruppamento logico dei task. È caratterizzata da un identificatore, un `titolo` (vincolato a categorie specifiche), una `descrizione` e un indice di ordinamento (`posizioneB`).
- **ToDo:** Il task vero e proprio, ricco di metadati. Definisce concetti come il `titolo`, la `dataScadenza`, l'aspetto visivo (`coloreSfondo`, `immagine`), eventuali riferimenti esterni (`linkURLs`), lo `stato` di completamento e la `posizione`. Contiene inoltre un riferimento al `permesso` applicato per la condivisione.

- **TitoloBacheca (Enumeration):** Definisce i tre ambiti standard del dominio: `UNIVERSITA`, `LAVORO` e `TEMPO_LIBERO`.
- **PermessoCondivisione (Enumeration):** Regola l'accesso multi-utente distinguendo tra `SOLO_LETTURA` e `MODIFICA`.

4. Architettura della Soluzione

4.1 Diagramma UML dell'Applicativo

Il seguente diagramma descrive lo scheletro reale del codice Java, esponendo l'uso dei pattern architetturali scelti per risolvere le problematiche di interazione con l'utente e con il database.

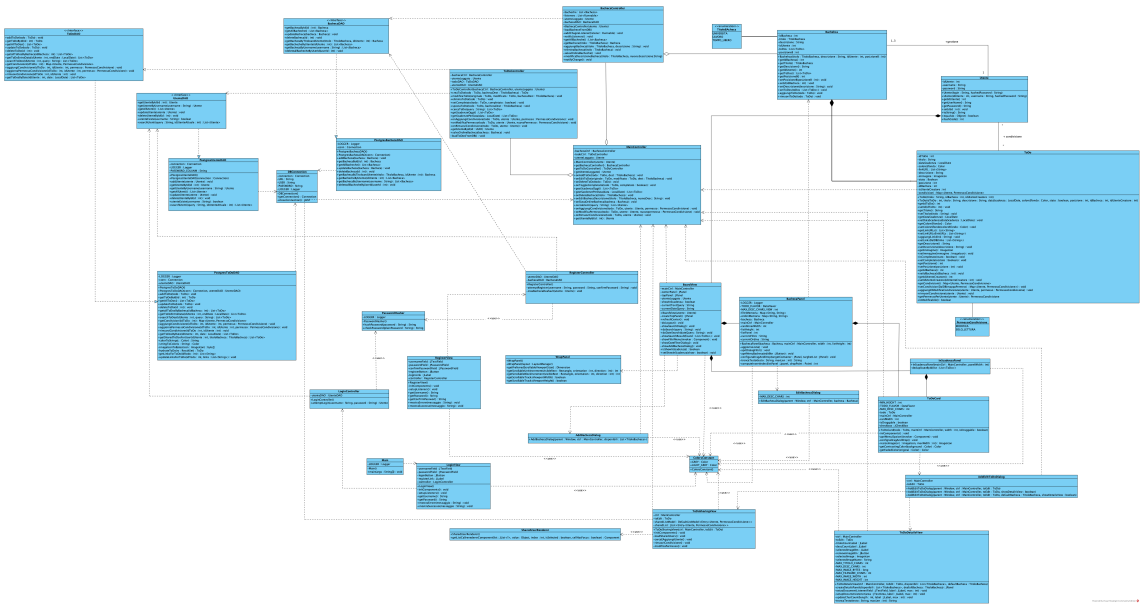

Figura 4.1: UML - Class Diagram Completo della Soluzione

4.2 Applicazione del Pattern MVC

Per garantire un software altamente manutenibile e scalabile, si è adottata una rigida separazione basata sul pattern Model-View-Controller:

- **View (Interfacce Grafiche):** Le classi appartenenti ai package `gui.views`, `gui.panels` e `gui.cards` gestiscono unicamente il layout e la cattura degli eventi. Istanziano oggetti visivi ma non calcolano dati e ignorano l'esistenza del database.

- **Controller (Coordinatori):** Il sistema è orchestrato da un `MainController` che agisce come Facade. Esso aggrega i controller specifici (`ToDoController` e `BachecaController`), elabora l'input della View, applica la logica e dialoga con lo strato di persistenza.

- **Model (Dati e Regole):** Le classi del modello (come `ToDo.java`) incapsulano lo stato e forniscono metodi intrinseci per validare i dati (es. regole sulle scadenze), isolando la business logic pura.

4.3 Pattern DAO e Singleton

La persistenza in PostgreSQL è isolata tramite il pattern Data Access Object (DAO). Sono state definite interfacce contrattuali (`ToDoDAO`, `BachecaDAO`) implementate concretamente dalle classi `Postgres...DAO`. I controller dialogano solo con le interfacce, garantendo un forte disaccoppiamento. Inoltre, la connessione al database è fornita in modo univoco ed efficiente dalla classe `DBConnection`, che agisce seguendo i principi del pattern Singleton.

5. Progettazione della Base di Dati

La persistenza dei dati dell'applicativo è gestita tramite PostgreSQL. Lo schema fisico del database è stato progettato per mappare in modo ottimizzato le classi del modello, garantendo l'integrità referenziale attraverso l'uso intensivo di chiavi esterne e vincoli di cancellazione a cascata.

5.1 Schema Logico Relazionale

Di seguito si riporta la struttura delle relazioni e i vincoli implementati:

- **UTENTE**(id_utente, username, password)
  Vincoli: username UNIQUE.
- **BACHECA**(id_bacheca, titolo, descrizione, posizioneB, *id_utente**)
  FK: id_utente references UTENTE(id_utente) ON DELETE CASCADE
- **TODO**(id_todo, titolo, descrizione, data_scadenza, colore_sfondo, immagine, stato, posizione, *id_bacheca**, *id_utente_creatore**)
  FK: id_bacheca references BACHECA(id_bacheca) ON DELETE CASCADE
  FK: id_utente_creatore references UTENTE(id_utente) ON DELETE CASCADE
- **TODO_LINKS**(id_link, url, *id_todo**)
  FK: id_todo references TODO(id_todo) ON DELETE CASCADE
- **TODO_CONDIVISIONE**(id_condivisione, permesso, *id_todo**, *id_utente**)
  FK: id_todo references TODO(id_todo) ON DELETE CASCADE
  FK: id_utente references UTENTE(id_utente) ON DELETE CASCADE
  Vincoli: UNIQUE(id_todo, id_utente)

5.2 Analisi delle Scelte Architetturali lato DB

Lo schema presenta due scelte progettuali di rilievo:

1. **ON DELETE CASCADE diffuso:** Traduce in SQL la relazione UML di Composizione tra Bacheca e ToDo. La cancellazione di una bacheca o di un utente "ripulisce" in automatico il database da tutti i ToDo, i link e le condivisioni figlie, prevenendo la formazione di dati orfani.

2. **Vincolo UNIQUE su Condivisioni:** La chiave univoca composita sulla tabella associativa `TODO_CONDIVISIONE` previene inconsistenze logiche. Evita che lo stesso utente possa essere invitato due volte al medesimo ToDo con permessi in conflitto, rispecchiando esattamente la struttura dati `Map<Utente, PermessoCondivisione>` utilizzata nel backend Java.

6. CRC Cards e Relazioni

Le CRC (Class-Responsibility-Collaboration) Cards definiscono formalmente le responsabilità di ogni componente del sistema e le interazioni necessarie per portare a termine i propri compiti.

6.1 CRC Cards del Dominio

Classe	**Responsabilità**	**Collaboratori**
Utente	Mantenere le informazioni di accesso. Essere assegnatario di Bacheche e ToDo.	Bacheca, ToDo
Bacheca	Fare da contenitore categorizzato per una serie di attività. Gestire l'ordine dei task.	Utente, ToDo, TitoloBacheca
ToDo	Gestire lo stato di un task. Mantenere le scadenze temporali. Tracciare i permessi degli utenti ospiti.	Utente, Bacheca, PermessoCondivisione

Tabella 6.1: CRC Cards per le Entità del Dominio

6.2 CRC Cards della Soluzione Architetturale (MVC/DAO)

Classe	Responsabilità	Collaboratori
MainController	Agire da Facade. Mantenere la sessione (utente loggato). Smistare richieste verso i sotto-controller.	ToDoController, BachecaController, View
ToDoController	Eseguire la logica di business sui ToDo. Verificare i permessi. Comandare i salvataggi ai DAO.	ToDo, ToDoDAO, UtenteDAO
PostgresToDoDAO	Implementare le query CRUD specifiche per PostgreSQL. Eseguire join su condivisioni e link.	DBConnection, ToDo, UtenteDAO
DBConnection	Fornire una connessione stabile e univoca al database JDBC.	Tutti i DAO concreti
ToDoCard	Componente UI atomico. Visualizzare i dati del ToDo, bloccare l'interazione se scaduto.	ToDo, MainController

Tabella 6.2: CRC Cards per Controller, View e DAO

6.3 Analisi Dettagliata delle Relazioni UML

Il diagramma UML dell'applicazione presenta legami strutturali precisi scelti per garantire un forte incapsulamento:

- **Bacheca → ToDo (Composizione):** Esprime un contenimento vitale. Nel dominio del sistema, un ToDo esiste solo ed esclusivamente all'interno di una Bacheca.
- **Utente ↔ ToDo (Associazione Multipla):** Risolve un doppio legame. Un utente è padrone (creatore) dei suoi task. Allo stesso tempo, può ospitare o essere ospitato su N task regolati dalla classe `PermessoCondivisione`.

- **MainController → Sotto-Controller (Aggregazione):** Il `MainController` è composto logicamente dai controller specifici. Si usa l'aggregazione (rombo vuoto) e non la composizione poiché i sotto-controller mantengono responsabilità e cicli di esecuzione propri.
- **Postgres...DAO → Interfacce DAO (Realizzazione):** Obbliga la classe concreta a rispettare i metodi CRUD astratti (linea tratteggiata), garantendo il paradigma di programmazione per interfacce.

7. Dettagli Implementativi (Listings)

Per dimostrare la solidità ingegneristica dell'applicativo, in questo capitolo si esaminano estratti di codice reale che evidenziano l'uso di algoritmi avanzati, API native di Java Swing e rigorose regole di business.

7.1 Regole di Business Temporali (24h Rule)

Per conferire realismo all'applicativo, si è imposto un vincolo di integrità: i ToDo non possono essere modificati all'infinito. È stato implementato un algoritmo nel Model che congela l'entità trascorse le 24 ore dalla scadenza. La GUI interroga questo metodo e inibisce strutturalmente le azioni dell'utente in caso di esito negativo.

```java
/**
 * Verifica se il ToDo e' ancora modificabile.
 * Concede 24 ore di tolleranza dopo la data di scadenza.
 * @return true se modificabile, false se bloccato.
 */
public boolean isModificabile() {
    if (this.dataScadenza == null) {
        return true; // Senza scadenza e' sempre modificabile
    }

    // Bloccato se la data odierna e' successiva alla scadenza + 1 giorno
    return !LocalDate.now().isAfter(this.dataScadenza.plusDays(1));
}
```

Listing 7.1: Metodo di validazione temporale nel Model (ToDo.java)

7.2 Calcolo Dinamico del Contrasto Cromatico

Il sistema permette la personalizzazione del colore di sfondo delle card. Per prevenire problemi di leggibilità (es. testo nero su sfondo blu scuro), è stata implementata una formula matematica basata sulla luminanza percepita per determinare dinamicamente il colore del font (bianco o nero).

```java
/**
 * Calcola il colore del testo in base alla luminosita' dello sfondo
```

```
 * per garantire il massimo contrasto e leggibilita'.
 */
private Color getContrastingColor(Color background) {
    if (background == null) return Color.BLACK;

    // Formula per la luminanza percepita standard: 0.299*R + 0.587*G + 0.114*B
    double luminance = 0.299 * background.getRed()
                     + 0.587 * background.getGreen()
                     + 0.114 * background.getBlue();

    // Se la luminanza e' bassa (< 128), lo sfondo e' scuro -> testo Bianco
    return luminance < 128 ? Color.WHITE : Color.BLACK;
}
```

Listing 7.2: Algoritmo di luminanza nella UI (ToDoCard.java)

7.3 Interattività Drag & Drop con TransferHandler

L'applicativo supera le classiche interazioni "point-and-click" integrando il Drag & Drop nativo di Java Swing. Ogni `ToDoCard` esporta se stessa come un oggetto trasferibile, permettendo all'utente di riorganizzare fisicamente i task nelle bacheche.

```
// Inizializzazione del DataFlavor univoco per l'oggetto ToDo
private static final DataFlavor TODO_FLAVOR;
static {
    try {
        TODO_FLAVOR = new DataFlavor(DataFlavor.javaJVMLocalObjectMimeType
                                     + ";class=" + ToDo.class.getName());
    } catch (ClassNotFoundException e) {
        throw new RuntimeException(e);
    }
}

// All'interno del costruttore: abilitazione del trascinamento
if (draggable && canEdit) {
    setTransferHandler(new TransferHandler() {
        @Override
        protected Transferable createTransferable(JComponent c) {
            return new Transferable() {
                @Override
```

```java
                public DataFlavor[] getTransferDataFlavors() {
                    return new DataFlavor[]{TODO_FLAVOR};
                }
                @Override
                public boolean isDataFlavorSupported(DataFlavor flavor) {
                    return TODO_FLAVOR.equals(flavor);
                }
                @Override
                public Object getTransferData(DataFlavor flavor) {
                    return todo; // Esporta l'oggetto Model reale
                }
            };
        }
        @Override
        public int getSourceActions(JComponent c) {
            return MOVE;
        }
    });
}
```

Listing 7.3: Configurazione del Drag & Drop (ToDoCard.java)

7.4 Gestione dei Permessi e Sicurezza lato View

L'estrazione e il controllo dei permessi di condivisione avvengono interrogando la mappa associativa racchiusa nel Model. La View si adatta disattivando elementi cruciali se l'utente possiede i diritti di sola lettura o non è l'autore.

```java
/**
 * Recupera il livello di permesso associato a un determinato utente.
 * @return PermessoCondivisione associato, o null se non ha accesso.
 */
public PermessoCondivisione getPermessoPerUtente(Utente utente) {
    return utente != null ? condivisioni.get(utente) : null;
}
```

Listing 7.4: Estrazione permessi associativi (ToDo.java)

Questi frammenti di codice certificano una forte aderenza ai principi dell'Ingegneria del Software (SOLID, Single Responsibility) applicati allo sviluppo grafico e logico dell'intera piattaforma.

8. Conclusioni

8.1 Analisi dei Risultati

Il software "Gestore ToDo" è stato completato nel pieno rispetto dei requisiti architetturali e funzionali richiesti dalla traccia, superandoli in termini di interattività e gestione dei permessi. L'applicazione si dimostra resiliente, scalabile e in grado di gestire flussi complessi come l'ordinamento grafico e la mutazione dinamica dei permessi collaborativi.

8.2 Qualità del Codice e Manutenibilità

Il rispetto rigoroso dello standard Object-Oriented è testimoniato dalla netta separazione delle responsabilità:

- La View (Swing) non esegue mai direttive SQL e non calcola vincoli.
- Il Controller orchestra senza inquinare le classi modello.
- Il Database (PostgreSQL) garantisce coerenza e isolamento transazionale con query preparate (`PreparedStatement`) per prevenire la SQL Injection.

Inoltre, le pratiche di ingegneria del software sono state rispettate bonificando il repository dal "dead code" in disuso e generando una documentazione Javadoc strutturata e priva di warning.

8.3 Prospettive di Sviluppo Futuro

La solidità dei layer architetturali (grazie al disaccoppiamento del pattern DAO) rende l'applicazione predisposta a future espansioni a basso costo ingegneristico. In un successivo ciclo di rilascio, l'infrastruttura dati già esistente potrebbe supportare agevolmente l'aggiunta di dashboard di statistiche di produttività, la storicizzazione in archivi delle attività completate e l'integrazione con API esterne per notifiche via mail.